



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

Laboratorio S07

Mexi Banco: de un monolito a servicios que se encuentran solos

Nombre del alumno:	Yukioayax Canek Gabriel Hernández
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre en curso:	2027-1

Ciudad Universitaria, CDMX

Índice

1. Introducción	2
2. Objetivo	2
3. Desarrollo	2
3.1. Paso 1: La historia (Hora: 20:00)	2
3.2. Paso 2: Directorio (Hora: 20:37)	3
3.3. Paso 3: Gateway (Hora: 20:54)	4
3.4. Pasos 4 y 5: Copias y caída (Hora: 21:54, 21:58 y 22:06)	5
3.5. Tarea: que transferencia también reparta (Hora: 22:19 y 22:20)	7
4. Resultados	9
5. Análisis de Resultados	9
6. Conclusiones	9

1 Introducción

El presente reporte documenta el desarrollo del Laboratorio S07, cuyo objetivo central es evolucionar la arquitectura de Mexi Banco desde un monolito hacia un sistema de servicios distribuidos que se descubren y se comunican entre sí sin direcciones escritas a mano; se implementó un directorio de servicios (Eureka), un API Gateway como puerta única de entrada, y un balanceador de carga que reparte peticiones entre múltiples réplicas de un mismo servicio.

2 Objetivo

Comprender los mecanismos de descubrimiento de servicios (*service discovery*), enrutamiento centralizado mediante un API Gateway y balanceo de carga del lado del cliente, aplicados a un sistema bancario compuesto por cinco servicios independientes (cuenta, movimiento, transferencia, notificación y SPEI), cada uno con su propia base de datos.

3 Desarrollo

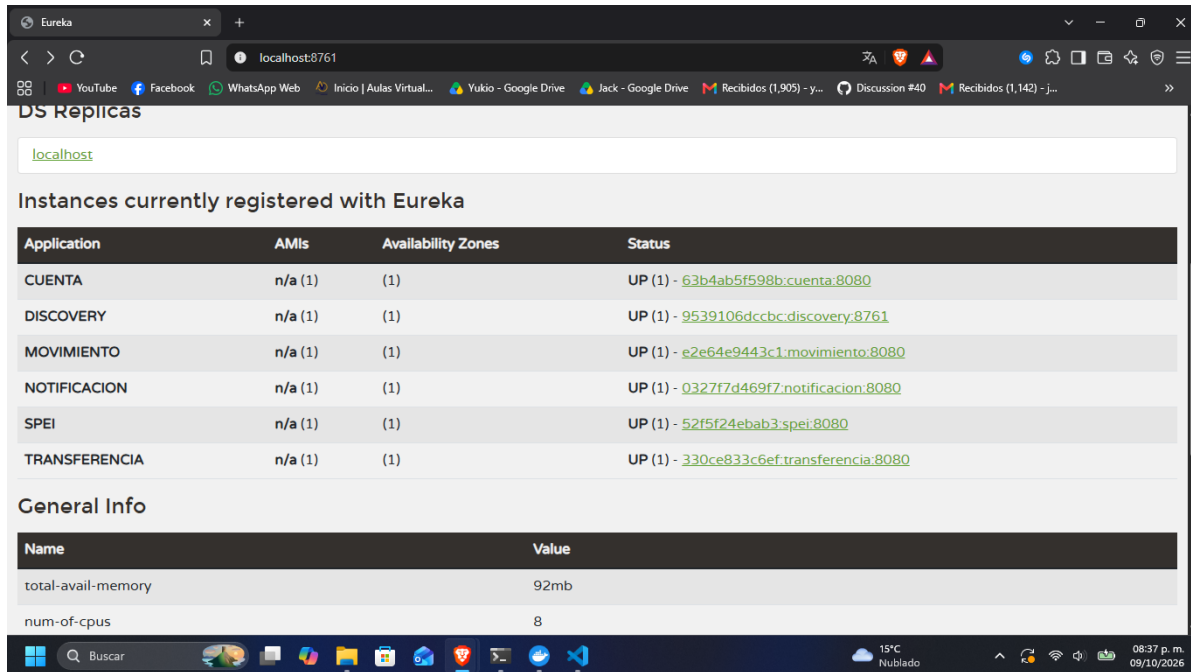
3.1 Paso 1: La historia (Hora: 20:00)

En el monolito, una transferencia era una sola llamada a método dentro de una transacción `@Transactional`: `TransferenciaService` le pedía a `CuentaService` que cargara a una cuenta y abonara a la otra; si el abono fallaba, la base deshacía también el cargo, o pasaba todo o no pasaba nada.

Al partir el banco en cinco servicios (cuenta, movimiento, transferencia, notificación y SPEI), cada uno es un programa aparte con su propia base de datos; el código de transferir casi no cambió, pero ahora `cuentas` ya no es un objeto en memoria, es un cliente HTTP que hace una petición a otro proceso. Ya no hay una transacción única que cubra el cargo y el abono, ahora son dos servicios y dos bases de datos distintas.

Eso rompió la garantía de “todo o nada”; si el abono falla después de que el cargo ya se aplicó, el dinero se mueve aunque la respuesta diga error, es el problema que vimos en el laboratorio: la transferencia contestó timeout pero el dinero sí se había movido.

3.2 Paso 2: Directorio (Hora: 20:37)



DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 63b4ab5f598b:cuenta:8080
DISCOVERY	n/a (1)	(1)	UP (1) - 9539106dccbcdiscovery:8761
MOVIMIENTO	n/a (1)	(1)	UP (1) - e2e64e9443c1movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 0327f7d469f7notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 52f5f24ebab3spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 330ce833c6eftransferencia:8080

General Info

Name	Value
total-avail-memory	92mb
num-of-cpus	8

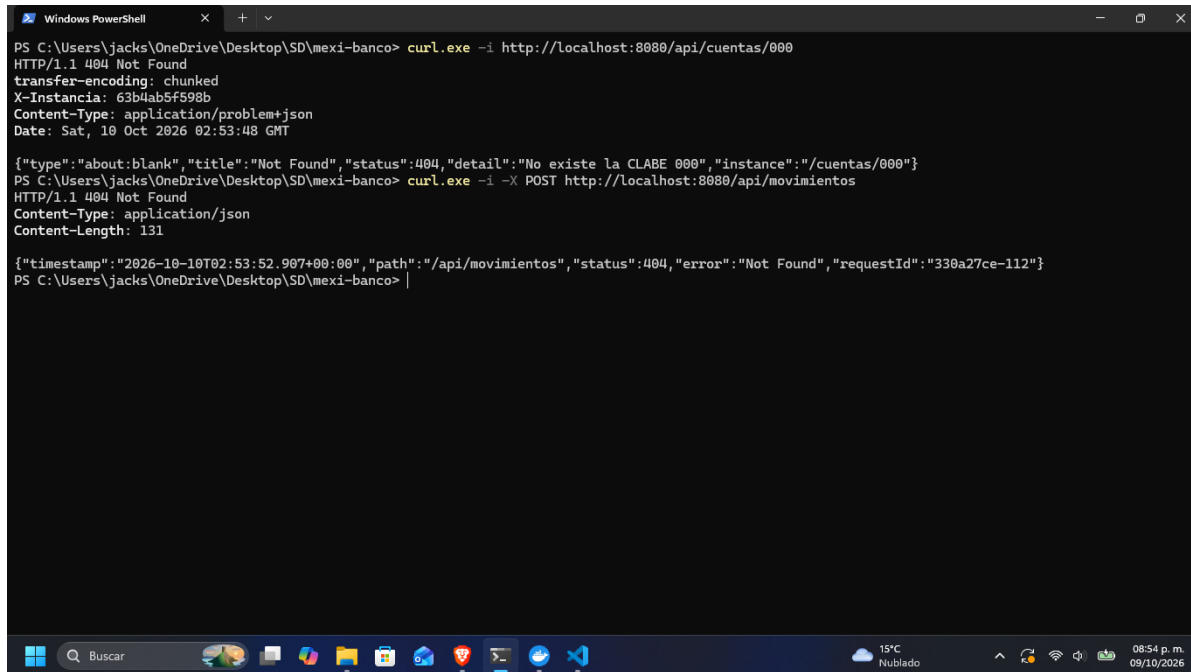
Figura 1: Tablero de Eureka mostrando los 7 servicios registrados con estatus UP (1).

Antes de encender Eureka, el cliente veía cinco puertas distintas (8081 a 8085) y cada servicio tenía la dirección del otro escrita a mano en el `docker-compose`; abrir `http://localhost:8761` no cargaba nada.

Después de completar el TODO H1 (anotación `@EnableEurekaServer` en `DiscoveryApplication.java`, las líneas `register-with-eureka` y `fetch-registry` en false en `application.yml`, y descomentar la dependencia de Eureka en los `pom.xml` de los cinco servicios), el tablero de Eureka mostró los cinco servicios registrados por nombre, cada uno diciendo UP (1): una copia viva.

El directorio resuelve el problema de saber qué servicios hay y dónde están, sin escribir direcciones a mano; cada servicio, al arrancar, se anota con su nombre y su dirección IP, después manda un *heartbeat* cada 5 segundos, y si el directorio deja de recibir *heartbeats* durante el plazo (15 segundos en el lab), borra al servicio de la agenda; quien quiera llamar a `cuenta` le pregunta al directorio y recibe la lista de direcciones vigentes.

3.3 Paso 3: Gateway (Hora: 20:54)



```
Windows PowerShell
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> curl.exe -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 63b4ab5f598b
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 02:53:48 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> curl.exe -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 131

{"timestamp":"2026-10-10T02:53:52.907+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"330a27ce-112"}
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco>
```

Figura 2: Pruebas del Gateway con curl: 404 con X-Instancia y 404 sin X-Instancia.

Antes de completar el TODO H2, cada servicio tenía su propia puerta hacia afuera (8081 a 8085); el cliente tenía que conocer cinco direcciones y cualquiera podía llamar a operaciones internas como `POST /movimientos`.

Un API Gateway resuelve los dos problemas: es la única entrada para los clientes, en el puerto 8080, y según la ruta de cada petición decide a qué servicio mandarla; lo que no tenga regla no pasa.

Completé las rutas en `gateway/src/main/resources/application.yml`: cinco rutas con la forma `uri: lb://<servicio>`, predicados `Path` y `Method`, y filtro `StripPrefix=1` que quita `/api` antes de reenviar; después descomenté el servicio `gateway` en `docker-compose.yml` y lo levanté con `docker compose up -d -build gateway`, recargué `localhost:8761` y apareció **GATEWAY** como sexto renglón.

Al probar:

- `GET http://localhost:8080/api/cuentas/000` dio 404 y SÍ trae `X-Instancia`: el gateway encontró la ruta, le preguntó al directorio, la petición llegó a cuenta y cuenta dijo “no existe la CLABE 000”.
- `POST http://localhost:8080/api/movimientos` dio 404 y NO trae `X-Instancia`: el gateway lo paró porque ninguna ruta acepta `POST` a `movimientos`; ese endpoint sigue abierto dentro de la red de Compose (donde lo usan los servicios), pero por la puerta ya no se alcanza.

3.4 Pasos 4 y 5: Copias y caída (Hora: 21:54, 21:58 y 22:06)

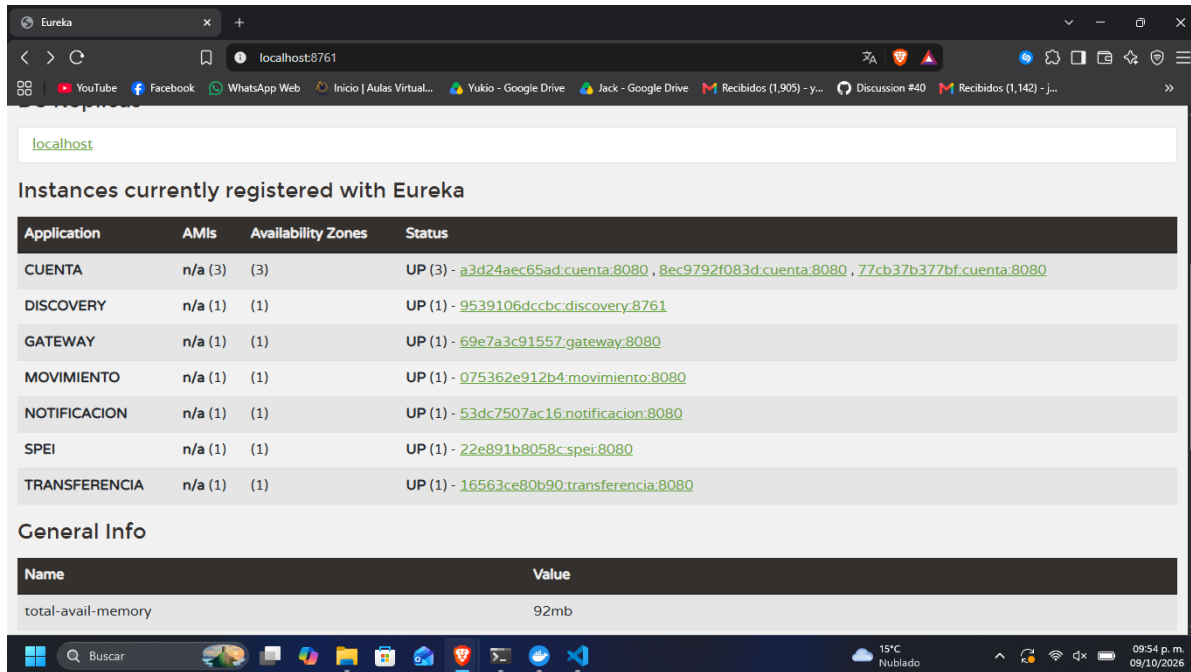


Figura 3: Tablero de Eureka mostrando CUENTA con UP (3) y tres IDs distintos.

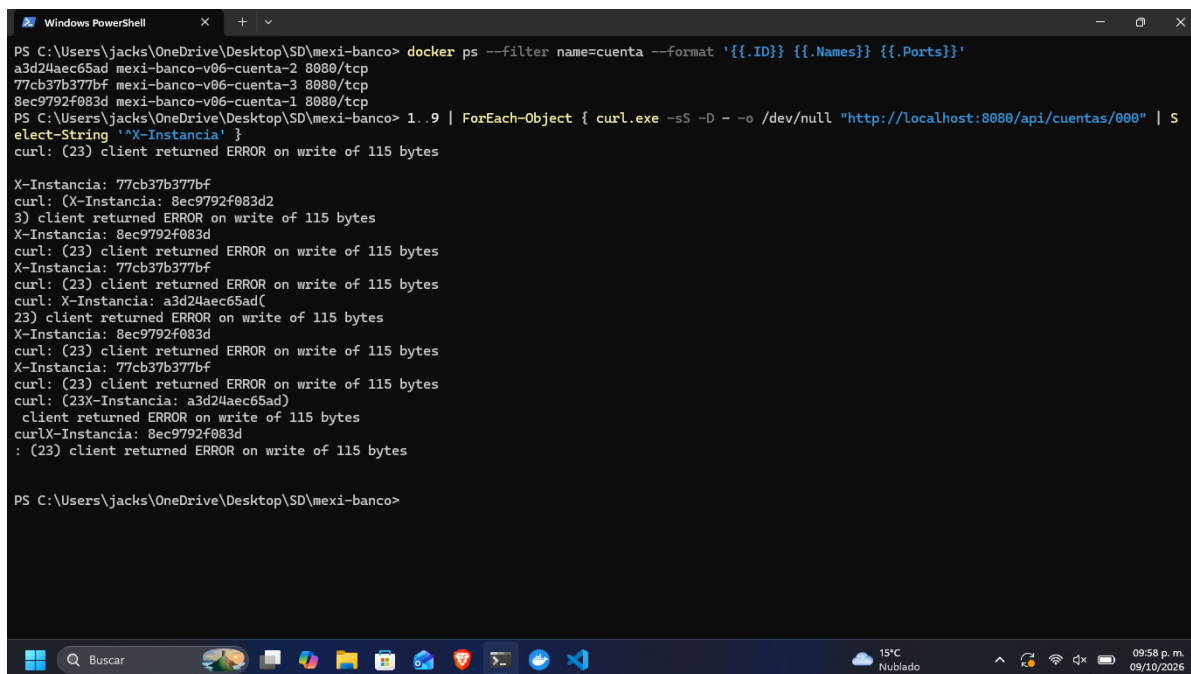


Figura 4: Bucle de 9 consultas mostrando el reparto round robin entre las 3 réplicas.

```

PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> $LAB_CLABE = "S07" + (Get-Date -Format "HHmmss")
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> $body = '{"clabe":"' + $LAB_CLABE + "', "titular":"S07", "saldoInicial":0}'
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> $response = curl.exe -s -X POST http://localhost:8080/api/cuentas -H "Content-Type: application/json" -d $body
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> Write-Host "Cuenta creada: $LAB_CLABE"
Cuenta creada: S07220415
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> Write-Host "Respuesta: $response"
Respuesta: {"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/cuentas"}
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> 1..60 | ForEach-Object { $time = Get-Date -Format "HH:mm:ss"; $code = curl.exe -s -o NUL -w "%{http_code}" "http://localhost:8080/api/cuentas/$LAB_CLABE"; Write-Host "$time $code"; Start-Sleep -Seconds 1 }
22:04:22 404
22:04:24 404
22:04:25 404
22:04:26 404
22:04:27 404
22:04:28 404
22:04:29 404
22:04:30 404
22:04:31 404
22:04:32 404
22:04:33 404
22:04:34 404
22:04:35 404
22:04:37 404
22:04:38 404
22:04:39 404
22:04:40 404
22:04:41 404
22:04:44 404
22:04:45 404
22:04:46 404
22:04:48 404
22:04:49 404
22:04:50 404
22:04:51 404

```

Figura 5: Bucle de 60 consultas monitoreando la caída de la copia 1.

```

PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> Get-Date -Format "22:04:40"
22:04:40
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 8ec9792f083d81457668e867cec2a709e42b90bba0adb65344b84c27620d3f
25 is not running
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 8ec9792f083d81457668e867cec2a709e42b90bba0adb65344b84c27620d3f
25 is not running
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 8ec9792f083d81457668e867cec2a709e42b90bba0adb65344b84c27620d3f
25 is not running
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 8ec9792f083d81457668e867cec2a709e42b90bba0adb65344b84c27620d3f
25 is not running
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 8ec9792f083d81457668e867cec2a709e42b90bba0adb65344b84c27620d3f
25 is not running
PS C:\Users\jacks\OneDrive\Desktop\SD\mexi-banco> |

```

Figura 6: Ejecución de docker kill sobre la copia 1 de cuenta.

Predicción antes de matar la copia (anotada antes del experimento):

“Van a fallar algunas consultas, no todas; durante unos 20-30 segundos van a aparecer errores 500/503, porque el gateway sigue mandando peticiones a la copia muerta hasta que el

directorio la expulsa; la copia desaparece del directorio después del plazo del heartbeat (15 segundos en este lab).”

Resultado observado:

Al matar la copia con `docker kill mexi-banco-v06-cuenta-1`, el sistema siguió respondiendo: el gateway enrutó a las 2 copias restantes; Eureka tardó entre 20 y 30 segundos en reflejar CUENTA UP (2). Durante esa ventana aparecieron errores 500/503 (los que por turno caían en la copia muerta); la hora del apagado fue 22:04:40 y la copia salió del directorio aproximadamente 30 segundos después.

Después de matar la copia, Eureka mostró CUENTA UP (2); el sistema siguió vivo y atendiendo con las dos copias restantes.

3.5 Tarea: que transferencia también reparta (Hora: 22:19 y 22:20)

```

UCRT64/c/Users/jacks/OneDrive/Desktop/SD/mexi-banco
jacks@Yukito UCRT64 ~
$ cd /c/Users/jacks/OneDrive/Desktop/SD/mexi-banco
jacks@Yukito UCRT64 ~/OneDrive/Desktop/SD/mexi-banco (trabajo-s07)
$ ./demo-v06.sh
docker compose logs cuenta | grep -E 'cargos|abonos'

== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ==
-> Listo en 15 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
DISCOVERY   1 instancia(s)
GATEWAY      1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPI         1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009160584101","titular":"Ana","saldoInicial":1000}'
{"timestamp":"2026-10-10T04:17:56.104+00:00","path":"/api/cuentas","status":504,"error":"Gateway Timeout","requestId":"bfd31ab2-883"}
[HTTP 504]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

== 2. Abrir la cuenta de Beto con 500 (misma puerta) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009160584102","titular":"Beto","saldoInicial":500}'
{"type":"about:blank","title":"Service Unavailable","status":503,"detail":"No se pudo contactar a otro servicio: Could not retrieve response status code: java.net.SocketTimeoutException: Read timed out","instance":"/cuentas"}
[HTTP 503]

== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160584101","claveDestino":"002180009160584102","monto":200}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160584101","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-2 GET /cuentas/002180009160584101 atendida por a3d24aec65ad
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

== 4. Consultar esa transferencia ==
$ curl http://localhost:8080/api/transferencias/
{"type":"about:blank","title":"Not Found","status":404,"detail":"No static resource transferencias.", "instance":"/transferencias/"}
[HTTP 404]

== 5. Saldos: el dinero no se crea ni se pierde ==
$ curl http://localhost:8080/api/cuentas/002180009160584101
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160584101","instance":"/cuentas/002180009160584101"}
[HTTP 404]
$ curl http://localhost:8080/api/cuentas/002180009160584102

```

Figura 7: Salida del `./demo-v06.sh`: secciones 0 a 4 (espera del directorio, apertura de cuentas, transferencia, consulta y saldos).

```

UCRT64/c:/Users/jacks/OneDrive/Desktop/SD/mexi-banco

--- 5. Saldos: el dinero no se crea ni se pierde ---
$ curl http://localhost:8080/api/cuentas/002180009160584101
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160584101","instance":"/cuentas/002180009160584101"}
[HTTP 404]
$ curl http://localhost:8080/api/cuentas/002180009160584102
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160584102","instance":"/cuentas/002180009160584102"}
[HTTP 404]
awk: cmd. line:1: BEGIN { printf "%2f", + }
^ syntax error
awk: cmd. line:1:
^ syntax error
-> Ana + Beto = (antes: 1000 + 500 = 1500). Esperado: 800 y 700

--- 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ---
$ curl http://localhost:8080/api/movimientos?clabe=002180009160584101
[{"id":1,"clabeCuenta":"002180009160584101","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T04:18:02.673187Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009160584102
[]
[HTTP 200]

--- 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ---
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160584101","bancoDestino":"BANCO-X","clabeDestino":"012180000000000009","monto":50}'
{"timestamp":"2026-10-10T04:18:50.802+00:00","path":"/api/spei","status":504,"error":"Gateway Timeout","requestId":"365f7084-904"}
[HTTP 504]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91605841' -d '{"claveOrigen":"002180009160584101","bancoDestino":"BANCO-X","clabeDestino":"012180000000000009","monto":50}'
{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya hay un SPEI en vuelo con esta Idempotency-Key","instance":"/spei"}
[HTTP 409]
-> mismo id (->): no se cobro dos veces. Saldo de Ana: (esperado 750)

--- 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ---
consulta 1 X-Instancia: a3d24aec65ad (mexi-banco-v06-cuenta-2)
consulta 2 X-Instancia: 8ec9792f083d (mexi-banco-v06-cuenta-1)
consulta 3 X-Instancia: 77cb37b377bf (mexi-banco-v06-cuenta-3)
consulta 4 X-Instancia: a3d24aec65ad (mexi-banco-v06-cuenta-2)
consulta 5 X-Instancia: 8ec9792f083d (mexi-banco-v06-cuenta-1)
consulta 6 X-Instancia: 77cb37b377bf (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: 8ec9792f083d (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: a3d24aec65ad (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 77cb37b377bf (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 0 veces se repitió la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

--- 9. Los endpoints internos no tienen puerta ---
$ curl -X POST http://localhost:8080/api/cuentas/002180009160584101/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T04:18:50.802+00:00","path":"/api/cuentas/002180009160584101/abonos","status":404,"error":"Not Found","requestId":"d2dc3778-922"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido
cuenta-2 | 2026-10-10T04:18:47.579Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160584101/cargos atendida por a3d24aec65ad
jacks@vukio UCRT64 ~/OneDrive/Desktop/SD/mexi-banco (trabaja-s07)
$ ./demo-v06.sh

```

Figura 8: Salida del ./demo-v06.sh: secciones 5 y 6 (saldos y estado de cuenta).

```

UCRT64/c:/Users/jacks/OneDrive/Desktop/SD/mexi-banco

--- 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ---
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91605841' -d '{"claveOrigen":"002180009160584101","bancoDestino":"BANCO-X","clabeDestino":"012180000000000009","monto":50}'
{"timestamp":"2026-10-10T04:18:50.802+00:00","path":"/api/spei","status":504,"error":"Gateway Timeout","requestId":"365f7084-904"}
[HTTP 504]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91605841' -d '{"claveOrigen":"002180009160584101","bancoDestino":"BANCO-X","clabeDestino":"012180000000000009","monto":50}'
{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya hay un SPEI en vuelo con esta Idempotency-Key","instance":"/spei"}
[HTTP 409]
-> mismo id (->): no se cobro dos veces. Saldo de Ana: (esperado 750)

--- 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ---
consulta 1 X-Instancia: a3d24aec65ad (mexi-banco-v06-cuenta-2)
consulta 2 X-Instancia: 8ec9792f083d (mexi-banco-v06-cuenta-1)
consulta 3 X-Instancia: 77cb37b377bf (mexi-banco-v06-cuenta-3)
consulta 4 X-Instancia: a3d24aec65ad (mexi-banco-v06-cuenta-2)
consulta 5 X-Instancia: 8ec9792f083d (mexi-banco-v06-cuenta-1)
consulta 6 X-Instancia: 77cb37b377bf (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: 8ec9792f083d (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: a3d24aec65ad (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 77cb37b377bf (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 0 veces se repitió la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

--- 9. Los endpoints internos no tienen puerta ---
$ curl -X POST http://localhost:8080/api/cuentas/002180009160584101/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T04:18:50.802+00:00","path":"/api/cuentas/002180009160584101/abonos","status":404,"error":"Not Found","requestId":"d2dc3778-922"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

--- 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ---
$ curl -X POST http://localhost:8080/api/transfencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160584101","claveDestino":"002180009160584199","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160584101","instance":"/transfencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transfencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160584101","claveDestino":"002180009160584102","monto":999999}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160584101","instance":"/transfencias","servicio":"cuenta"}
[HTTP 404]

--- 11. Los saldos no se movieron con los errores ---
awk: cmd. line:1: BEGIN { printf "%2f", + }
^ syntax error
awk: cmd. line:1:
^ syntax error
-> Ana (esperado 750) + Beto (esperado 700) = ; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido
cuenta-2 | 2026-10-10T04:18:47.579Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160584101/cargos atendida por a3d24aec65ad
jacks@vukio UCRT64 ~/OneDrive/Desktop/SD/mexi-banco (trabaja-s07)
$ ./demo-v06.sh

```

Figura 9: Salida del ./demo-v06.sh: secciones 7 a 9 (SPEI, round robin y endpoints internos).

En la T1 agregué a transferencia/src/main/java/mx/mexibanco/clientes/CientesHttp.java los dos imports (RestClientBuilderConfigurer y LoadBalanced) y el método @Bean @LoadBalanced que recibe el configurador y regresa configurador.configure(RestClient.builder()).

CientesHttp.java — Método @LoadBalanced

```
@Bean @LoadBalanced RestClient.Builder restClientBalanceado(RestClientBuilderCon
configurador) return configurador.configure(RestClient.builder());
```

En la T2 cambié las URLs en `docker-compose.yml`, dentro de `transferencia` → `environment`, para que llamen por nombre sin puerto: `http://cuenta`, `http://movimiento` y `http://notificacion`.

En la T3 reconstruí `transferencia` con `docker compose up -d -build transferencia` y corrí `./demo-v06.sh`; el script corrió completo: `transferencia` llamó a `http://cuenta` por nombre, el balanceador repartió entre las 3 réplicas (se ve en los logs: `cuenta-1`, `cuenta-2` y `cuenta-3` atendiendo cargos y abonos distintos), y los saldos finales quedaron en 800/700.

4 Resultados

Se logró desplegar exitosamente la arquitectura de servicios distribuidos con Mexi Banco: el directorio de Eureka registró los siete servicios (cinco de negocio, gateway y discovery), el Gateway enrutó correctamente las peticiones públicas y bloqueó las internas, las tres réplicas de cuenta se registraron y el balanceador repartió las peticiones en round robin, y al matar una copia el sistema se mantuvo disponible con las dos restantes.

5 Análisis de Resultados

La práctica evidenció la importancia del descubrimiento de servicios para desacoplar las direcciones físicas de los servicios; el Gateway demostró ser una pieza clave de seguridad y organización, ya que sin él cada servicio tendría que exponer su propio puerto y no habría control sobre qué operaciones se permiten desde afuera. El balanceo de carga con `@LoadBalanced` permitió que `transferencia` llamara a `http://cuenta` por nombre y obtuviera una instancia viva del directorio, sin saber cuántas copias hay ni cuáles están activas; la caída simulada mostró la ventana de inconsistencia inherente a los sistemas distribuidos: el directorio tarda en detectar la falla (heartbeat cada 5s + plazo de 15s) y durante ese tiempo el Gateway sigue enviando peticiones a la copia muerta.

6 Conclusiones

El directorio resuelve saber qué servicios hay y dónde están, sin escribir direcciones a mano; el gateway resuelve tener una sola puerta con reglas de qué se puede llamar desde afuera; el balanceador resuelve repartir entre copias y dejar de mandar a las que mueren. Cuando murió una copia vi errores 500/503 durante unos 20-30 segundos, porque el directorio tarda en enterarse: el heartbeat es cada 5 segundos y el plazo de expiración es 15 segundos, más el refresh del balanceador (5 segundos) y el caché del gateway (5 segundos).